Uniwersytet Kazimierza Wielkiego w Bydgoszczy

**Wydział Informatyki**

ul. Kopernika 1, 85-074 Bydgoszcz

informatyka@ukw.edu.pl

# Sprawozdanie z zadania Z2

Baza wiedzy w JSON i silnik wnioskujący w przód i wstecz · Laboratorium nr 2 · 2026/2027

| | |
|---|---|
| **Przedmiot** | Inżynieria wiedzy i maszynowe uczenie · Informatyka, II rok |
| **Autor · nr albumu · grupa** | Piotr Dalewski · 104125 · gr. 1 |
| **Nazwa systemu (dziedzina)** | System wspomagający alokację PIP i zarządzanie energią statku w Elite Dangerous |
| **Repozytorium · commit** | https://github.com/Xaaki/PD_Stu26-27_IWiSE · be7a4fe |
| **Narzędzia AI** | Gemini (Flash): Pomoc w opracowywaniu reguł. |
| **Prowadzący** | mgr inż. Marek Andryszczyk |
| **Data oddania** | 2026-11-08 (termin: spotkanie 3) |

**Wymagania. Silnik z TODO 1–4 (autotest 5/5); baza z min. 20 regułami i łańcuchem na 3 poziomach atrybutów (wejście → wniosek pośredni → cel), metafora w każdej regule, coto przy decyzjach końcowych; min. 5 własnych testów, w tym wartość graniczna i konflikt reguł.** W repozytorium: se/silnik.py, wiedza/moja_dziedzina.json, se/test_moja_dziedzina.py. Oddanie: IWiMU_Z2_Nazwisko_Imie.pdf w zadaniu Z2 w MS Teams, termin: spotkanie 3.

## 1. Silnik wnioskujący

Tab. 1. Uzupełnione fragmenty silnika. Źródło: opracowanie własne.

| TODO | Funkcja | Rozwiązanie |
|---|---|---|
| 1 | regula_spelniona | return all(warunek_spelniony(w, fakty) for w in regula["jesli"]) |
| 2 | wnioskuj_w_przod | if atrybut not in fakty and regula_spelniona(regula, fakty): fakty[atrybut] = wartosc; slad.append(regula["id"]); zmiana = True |
| 3 | wnioskuj_wstecz | if not reguly: fakty[cel] = zapytaj(cel, powod); return fakty[cel] |
| 4 | wnioskuj_wstecz | znana = wnioskuj_wstecz(baza, atrybut, fakty, zapytaj, slad, regula); if znana is None or not OPERATORY[op](znana, wartosc): spelniona = False; break |

```
xixi@localhost:~/Desktop/studia/Inżynieria wiedzy i systemy expertowe>
python3 se/test_silnik.py
[OK ] {'temperatura': 18, 'pora_dnia': 'rano', 'dodatek': 'brak',
'kofeina_ok': 'tak'} -> w przód: espresso, wstecz: espresso, pytań: 4
[OK ] {'temperatura': 30, 'pora_dnia': 'popołudnie', 'dodatek':
'czekolada', 'kofeina_ok': 'tak'} -> w przód: chocolate cold coffee,
wstecz: chocolate cold coffee, pytań: 4
[OK ] {'temperatura': 10, 'pora_dnia': 'wieczór', 'dodatek': 'mleko',
'kofeina_ok': 'tak'} -> w przód: latte bezkofeinowe, wstecz: latte
bezkofeinowe, pytań: 3
```

```
[OK ] {'temperatura': 25, 'pora_dnia': 'rano', 'dodatek': 'mleko',
'kofeina_ok': 'nie'} -> w przód: mrożona kawa bezkofeinowa, wstecz: mrożona
kawa bezkofeinowa, pytań: 2
[OK ] {'temperatura': 22, 'pora_dnia': 'popołudnie', 'dodatek': 'mleko',
'kofeina_ok': 'tak'} -> w przód: latte, wstecz: latte, pytań: 4

Zaliczone przypadki: 5/5
```

Listing 1. Wynik autotestu silnika. Źródło: opracowanie własne.

## 2. Baza wiedzy

Tab. 2. Atrybuty bazy wiedzy. Źródło: opracowanie własne.

| Atrybut | Typ | Wartości / zakres | Poziom |
|---|---|---|---|
| przeciwnik_obecny | wybor | Tak, Nie | wejście |
| typ_statku | wybor | `Bojowy`, `Exploracyjny`, `Górniczy` | wejście |
| stan_oslon_proc | liczba | [0, 100] | wejście |
| stan_wep | wybor | `Pusty`, `Optymalny` | wejście |
| dystans_do_celu | liczba | `[0, 10000]` | wejście |
| tryb | wniosek | `walka`, `ucieczka`, `spokojny` | wniosek pośredni |
| `decyzja` | wniosek | PIP_3SYS_1ENG_2WEP, PIP_3SYS_2ENG_1WEP, PIP_2SYS_0ENG_4WEP, PIP_2SYS_1ENG_3WEP, PIP_2SYS_2ENG_2WEP, PIP_2SYS_3ENG_1WEP, PIP_2SYS_4ENG_0WEP, PIP_4SYS_1ENG_1WEP, PIP_3SYS_3ENG_0WEP, PIP_1SYS_4ENG_1WEP, PIP_0SYS_4ENG_2WEP, PIP_1SYS_1ENG_4WEP. | cel |

Baza wiedzy zawiera łącznie 17 reguł (w tym 4 reguły ustalające wnioski pośrednie `P1`, `P2_1`, `P2_2`, `P3` oraz 13 reguł ustalających decyzje końcowe `R1`–`R15`, `R16_KRAŃCOWA_TARCZA`, `R17_KRAŃCOWY_DYSTANS`).

Przykład łańcucha reguł przechodzącego przez 3 poziomy atrybutów (dana wejściowa -> wniosek pośredni -> decyzja końcowa):

```
[
  {
    "id": "P1",
    "jesli": [
      ["przeciwnik_obecny", "==", "Tak"],
      ["typ_statku", "==", "Bojowy"]
    ],
    "to": ["tryb", "walka"],
```

```
    "metafora": "Obecność wroga i statek bojowy oznaczają konieczność
podjęcia natychmiastowej walki."
  },
  {
    "id": "R1",
    "jesli": [
      ["tryb", "==", "walka"],
      ["stan_oslon_proc", "<=", 25],
      ["stan_wep", "==", "Pusty"]
    ],
    "to": ["decyzja", "PIP_3SYS_1ENG_2WEP"],
    "metafora": "Padające tarcze i brak mocy broni wymagają priorytetu dla
SYS i częściowego ładowania WEP.",
    "coto": "Alokacja defensywna przy braku broni: 3 PIP-y SYS na tarcze, 1
PIP ENG na zwrot i 2 PIP-y WEP."
  }
]
```

```
> python se/sprawdz_format.py wiedza/moja_dziedzina.json
Format: bez błędów i uwag.

Wymagania Z2:
  [TAK] co najmniej 20 reguł (jest 21)
  [TAK] łańcuch co najmniej 3 poziomów atrybutów: wejście → wniosek
pośredni → cel (jest 3)
  [TAK] pole 'metafora' w każdej regule
  [TAK] pole 'coto' w każdej regule z decyzją końcową
```

Listing 2. Wynik sprawdzenia formatu bazy (część „Wymagania Z2”). Źródło: opracowanie własne.

## 3. Testy własnej bazy

Tab. 3. Przypadki testowe i wyniki obu trybów. Źródło: opracowanie własne.

| Nr | Rodzaj | Dane wejściowe | Oczekiwany | W przód | Wstecz |
|---|---|---|---|---|---|
| T1 | typowy | `przeciwnik_obecny=Tak, typ_statku=Bojowy, stan_oslon_proc=80, stan_wep=Pusty, dystans_do_celu=1200` | PIP_2SYS_0ENG_4WEP | PIP_2SYS_0ENG_4WEP | PIP_2SYS_0ENG_4WEP |
| T2 | typowy | `przeciwnik_obecny=Tak, typ_statku=Exploracyjny, stan_oslon_proc=100, stan_wep=Optymalny, dystans_do_celu=3000` | PIP_2SYS_4ENG_0WEP | PIP_2SYS_4ENG_0WEP | PIP_2SYS_4ENG_0WEP |
| T3 | granica | `przeciwnik_obecny=Tak, typ_statku=Bojowy, stan_oslon_proc=25, stan_wep=Pusty, dystans_do_celu=1000` | PIP_3SYS_1ENG_2WEP | PIP_3SYS_1ENG_2WEP | PIP_3SYS_1ENG_2WEP |
| T4 | brak wyniku | `przeciwnik_obecny=Nie, typ_statku=Niezdefinowany` | None | None | None |
| T5 | konflikt | `przeciwnik_obecny=Tak, typ_statku=Bojowy, stan_oslon_proc=0, stan_wep=Pusty` | PIP_3SYS_1ENG_2WEP | PIP_3SYS_1ENG_2WEP | PIP_3SYS_1ENG_2WEP |

## 4. Konsultacja w trybie wstecz

```
> python se/silnik.py
=== Wnioskowanie w przód ===
Wynik: PIP_3SYS_1ENG_2WEP | reguły: P1 -> R1

=== Wnioskowanie wstecz ===
Czy przeciwnik jest obecny na radarze? (Tak/Nie) Tak
Jaki jest typ Twój statku? (Bojowy/Exploracyjny/Górniczy) Bojowy
Jaki jest obecny poziom osłon w procentach (0-100)? 70
Jaka jest odległość do celu w metrach (0-10000)? 3000
Jaki jest stan kondensatora uzbrojenia (WEP)? (Pusty/Optymalny) Optymalny
Polecam: PIP_2SYS_3ENG_1WEP
Reguły: P1 -> R6
```

Listing 3. Konsultacja w trybie wstecz. Źródło: opracowanie własne.

1. Dlaczego system zadał te pytania i w tej kolejności:
   - **`przeciwnik_obecny` i `typ_statku`:** System rozpoczyna wnioskowanie od celu głównego `decyzja`. Pierwszą rozważaną hipotezą w bazie jest reguła R1,

której warunkiem jest wniosek pośredni `tryb == walka`. Aby go ustalić, silnik wywołuje się rekurencyjnie dla atrybutu `tryb`. Zwraca on regułę `P1`, która wymaga znajomości `przeciwnik_obecny` oraz `typ_statku`. System pyta o nie w pierwszej kolejności i wylicza wniosek pośredni `tryb = walka`.

- **`stan_oslon_proc`:** Po powrocie do sprawdzania reguły `R1` system pyta o stan osłon. Użytkownik podaje `70`, co sprawia, że warunek `stan_oslon_proc <= 25` z reguły `R1` nie jest spełniony (`False`). Reguła `R1` odpada (`break`), a system przechodzi do kolejnych reguł (`R2`, `R3`, `R4`, `R5` itd.).
- **`dystans_do_celu` i `stan_wep`:** Przechodząc do reguły `R6`, system weryfikuje warunek `dystans_do_celu > 1500`. Ponieważ wartość ta nie jest jeszcze znana w faktach, pyta o nią użytkownika (`3000`). Warunek zostaje spełniony. Na koniec pyta o `stan_wep` (`Optymalny`), co ostatecznie potwierdza regułę `R6` i kończy wnioskowanie wydaniem decyzji `PIP_2SYS_3ENG_1WEP`.

2. O co system nie zapytał i dlaczego:
   - System **nie zapytał o żaden z niepotrzebnych wariantów**, np. o wartości wchodzące w skład ścieżek ucieczki dla statków `Eksploracyjny` czy `Górniczy` (np. reguły `P2_1`, `P2_2`, `R7`, `R8`). Dzięki mechanizmowi wnioskowania wstecz pytania są generowane wyłącznie dla aktualnie weryfikowanej hipotezy, co eliminuje zbędną interakcję z użytkownikiem.

## 5. Strategia rozwiązywania konfliktów

W silniku wnioskującym przyjęto **strategię pierwszego dopasowania (kolejności reguł w pliku)**. Silnik przegląda listę reguł od góry do dołu. Gdy napotka pierwszą spełnioną regułę dla danego celu/wniosku, ustala jego wartość i dopisuje do faktów. Raz ustalony fakt nie jest nadpisywany w kolejnych przebiegach pętli (warunek atrybut not in fakty).

Przykład wpływu kolejności reguł na wynik z własnej bazy:

W pliku wiedza/moja_dziedzina.json zdefiniowano dwie konkurujące reguły defensywne dla sytuacji awaryjnej osłon:

- **R1**: stan_oslon_proc <= 25 -> decyzja = PIP_3SYS_1ENG_2WEP
- **R16_KRAŃCOWA_TARCZA**: stan_oslon_proc == 0 -> decyzja = PIP_4SYS_1ENG_1WEP

Dla danych wejściowych: stan_oslon_proc = 0, przeciwnik_obecny = Tak, typ_statku = Bojowy, stan_wep = Pusty:

1. **Stan obecny (R1 przed R16):** Warunek stan_oslon_proc <= 25 w regule R1 zostaje spełniony jako pierwszy. Silnik ustala decyzja = PIP_3SYS_1ENG_2WEP. Reguła R16 (mimo że bardziej szczegółowa) zostaje pominięta.

2. **Po zamianie kolejności (R16 przed R1):** Gdyby reguła R16_KRAŃCOWA_TARCZA została przesunięta wyżej w pliku JSON, silnik dopasowałby ją w pierwszej kolejności i zwrócił silniejszą alokację defensywną PIP_4SYS_1ENG_1WEP.

**Wniosek:** Kolejność zapisu reguł w pliku JSON stanowi element wiedzy dziedzinowej. Przetwarzanie od najbardzie szczegółowych (krytycznych) do ogólnych reguł jest kluczowe dla prawidłowego działania systemu ekspertowego.

## 6. Wnioski

1. **Oddzielenie wiedzy od mechanizmu wnioskowania:** Zapisanie reguł w rodzimym formacie JSON pozwala na zmianę logiki dziedziny (Elite Dangerous) bez ingerencji w kod źródłowy silnika Python (`silnik.py`).
2. Różnice i specyfika trybów wnioskowania:
   - **W przód:** Przetwarza cały dostępny zestaw faktów wejściowych naraz, wyliczając wszystkie możliwe wnioski pośrednie i końcowe (przydatne przy podpięciu pod czujniki lub gotowe formularze).
   - **Wstecz:** Pozwala na interaktywny dialog z użytkownikiem, dynamicznie wyznaczając podcele i zadając pytania wyłącznie o atrybuty wejściowe niezbędne do weryfikacji aktualnie sprawdzanej hipotezy.
3. **Znaczenie struktur łańcuchowych:** Wprowadzenie łańcucha 3 poziomów atrybutów (wejście -> wniosek pośredni `tryb` -> cel `decyzja`) pozwoliło na znaczną redukcję liczby reguł i wyeliminowanie powtarzających się warunków w liściach drzewa decyzyjnego.
4. **Ograniczenia rozwiązania:** System bazuje na prostej strategii rozwiązywania konfliktów (kolejność w pliku). W przypadku rozbudowy o setki reguł lepszym rozwiązaniem byłoby wprowadzenie priorytetów/wag reguł (np. parametr `salience` / pewność). Ponadto system nie obsługuje jeszcze operacji LUB bezpośrednio w warunku jednej reguły (wymaga to pisania osobnych reguł dla każdego wariantu).

## Bibliografia

[1] A. Wakulicz-Deja, A. Nowak-Brzezińska, M. Przybyła-Kasperek, R. Simiński, Systemy ekspertowe. Warszawa: Akademicka Oficyna Wydawnicza EXIT, 2018.

[2] Frontier Developments, Elite Dangerous Game Manual, URL: http://hosting.zaonce.net/elite/website/assets/ELITE-DANGEROUS-GAME-MANUAL.pdf (dostęp: 2026-11-07).

[3] Materiały do laboratorium nr 2 z przedmiotu *Inżynieria wiedzy i maszynowe uczenie*, Wydział Informatyki UKW w Bydgoszczy, 2026.

**Wydział Informatyki**

ul. Kopernika 1, 85-074 Bydgoszcz

informatyka@ukw.edu.pl

## Ocena (wypełnia prowadzący)

| Kryterium | Maks. | Pkt |
|---|---|---|
| Silnik: autotest 5/5 | 4 | |
| Baza: 20 reguł, 3 poziomy, oba tryby | 3 | |
| Własne testy (min. 5, w tym granica i konflikt) | 2 | |
| Konsultacja i strategia konfliktów | 1 | |
| Razem | 10 | |